American International University-Bangladesh (AIUB)

Department of Computer Science

Faculty of Science and Technology

Final Term Project Documentation

ADVANCE DATABASE MANAGEMENT SYSTEM

Semester: SUMMER 2024-2025
Section: [A]

Supervised By

Juena Ahmed Noshin

Topic Name :

Outdoor Sports Facility Management System

Student Details			
Student Name	**Student ID**	**Contributed Topic**	**Percentage%**
MD. SHADMAN SHAKIB ALAM	22-46262-1	Normalization, Design	25%
RIMON PRAMANIK	22-46957-1	Schema diagram, Design	25%
LAMIA TAHSIN	22-46345-1	PL/SQL, Design	25%
MARUF AHMEND	22-46354-1	Relational Algebra, Design	25%

Submission Date: September 16, 2025

Updated:

1. **NORMALIZATION:**

Relation 1: Can Manage (Staff Member Can Manage Event)

UNF:

Can_Manage(Staff_ID, First_Name, Last_Name, Department, Event_ID, Event_Name, Date, Rules)

1NF:

No multivalued attributes, so it remains

1. Staff_ID, First_Name, Last_Name, Department, Event_ID, Event_Name, Date, Rules

2NF:

Staff details depend on Staff_ID

Event details depend on Event_ID.

1. Staff_ID, First_Name, Last_Name, Department

2. Event_ID, Event_Name, Date, Rules

3. Staff_ID, Event_ID

3NF:

No Transitive dependencies.

1. Staff_ID, First_Name, Last_Name, Department

2. Event_ID, Event_Name, Date, Rules

3. Staff_ID, Event_ID

Table Creation:

Staff_Member(**Staff_ID (PK)**, First_Name, Last_Name, Department)

Event(**Event_ID (PK)**, Event_Name, Date, Rules)

Can_Manage(**Staff_ID(FK)**, **Event_ID(FK)**)

Relation 2: Participates (Event Participates in Team)

UNF:

Participates(Event_ID, Event_Name, Date, Rules, Team_ID, Team_Name, Sport_Type)

1NF:

No multivalued attributes, so it remains

1. Event_ID, Event_Name, Date, Rules, Team_ID, Team_Name, Sport_Type

2NF:

Event details depend on Event_ID.

Team details depend on Team_ID.

1. Event_ID, Event_Name, Date, Rules

2. Team_ID, Team_Name, Sport_Type

3. Event_ID, Team_ID

3NF:

No Transitive dependencies.

1. Event_ID, Event_Name, Date, Rules

2. Team_ID, Team_Name, Sport_Type

3. Event_ID, Team_ID

Table Creation:

Event(**Event_ID(PK)**, Event_Name, Date, Rules)

Team(**Team_ID(PK)**, Team_Name, Sport_Type)

Participates(**Event_ID(FK)**, **Team_ID(FK)**)

Relation 3: Belongs To (Player Belongs to Team)

UNF:

Belongs_To(Player_ID, First_Name, Last_Name, DOB, Phone_Number, Team_ID, Team_Name, Sport_Type)

1NF:

No multivalued attributes, so it remains

1. Player_ID, First_Name, Last_Name, DOB, Phone_Number, Team_ID, Team_Name, Sport_Type

2NF:

Player details depend on Player_ID.

Team details depend on Team_ID.

1. Player_ID, First_Name, Last_Name, DOB, Phone_Number

2. Team_ID, Team_Name, Sport_Type

3. Player_ID, Team_ID

3NF:

No Transitive dependencies.

1. Player_ID, First_Name, Last_Name, DOB, Phone_Number

2. Team_ID, Team_Name, Sport_Type

3. Player_ID, Team_ID

Table Creation:

Player(**Player_ID(PK)**, First_Name, Last_Name, DOB, Phone_Number)

Team(**Team_ID(PK)**, Team_Name, Sport_Type)

Belongs_To(**Player_ID(FK)**, **Team_ID(FK)**)

Relation 4: Has (Player Has Profile)

UNF:

Has(Player_ID, First_Name, Last_Name, DOB, Phone_Number, Profile_ID, Skill_Level, Achievements)

1NF:

No multivalued attributes, so it remains

1. Player_ID, First_Name, Last_Name, DOB, Phone_Number, Profile_ID, Skill_Level, Achievements

2NF:

Player details depend on Player_ID.

Profile details depend on Profile_ID.

1. Player_ID, First_Name, Last_Name, DOB, Phone_Number

2. Profile_ID, Skill_Level, Achievements

3. Player_ID, Profile_ID

3NF:

No Transitive dependencies.

1. Player_ID, First_Name, Last_Name, DOB, Phone_Number

2. Profile_ID, Skill_Level, Achievements

3. Player_ID, Profile_ID

Table Creation:

Player(**Player_ID(PK)**, First_Name, Last_Name, DOB, Phone_Number)

Profile(**Profile_ID(PK)**, Skill_Level, Achievements)

Has(**Player_ID(FK)**, **Profile_ID(FK)**)

Relation 5: Can Make (Team Can Make Booking)

UNF:

Can_Make(Team_ID, Team_Name, Sport_Type, Booking_ID, Start_Time, End_Time, Date, Status)

1NF:

No multivalued attributes, so it remains

1. Team_ID, Team_Name, Sport_Type, Booking_ID, Start_Time, End_Time, Date, Status

2NF:

Team details depend on Team_ID

Booking details depend on Booking_ID

1. Team_ID, Team_Name, Sport_Type

2. Booking_ID, Start_Time, End_Time, Date, Status

3. Team_ID, Booking_ID

3NF:

No Transitive dependencies.

1. Team_ID, Team_Name, Sport_Type

2. Booking_ID, Start_Time, End_Time, Date, Status

3. Team_ID, Booking_ID

Table Creation:

Team(**Team_ID(PK)**, Team_Name, Sport_Type)

Booking(**Booking_ID(PK)**, Start_Time, End_Time, Date, Status)

Can_Make(**Team_ID(FK)**, **Booking_ID(FK)**)

Relation 6: Makes (Player Makes Booking)

UNF:

Makes(Player_ID, First_Name, Last_Name, DOB, Phone_Number, Booking_ID, Start_Time, End_Time, Date, Status)

1NF:

No multivalued attributes, so it remains

1. Player_ID, First_Name, Last_Name, DOB, Phone_Number, Booking_ID, Start_Time, End_Time, Date, Status

2NF:

Player details depend on Player_ID

Booking details depend on Booking_ID

1. Player_ID, First_Name, Last_Name, DOB, Phone_Number

2. Booking_ID, Start_Time, End_Time, Date, Status

3. Player_ID, Booking_ID

3NF:

No Transitive dependencies.

1. Player_ID, First_Name, Last_Name, DOB, Phone_Number

2. Booking_ID, Start_Time, End_Time, Date, Status

3. Player_ID, Booking_ID

Table Creation:

Player(**Player_ID(PK)**, First_Name, Last_Name, DOB, Phone_Number)

Booking(**Booking_ID(PK)**, Start_Time, End_Time, Date, Status)

Makes(**Player_ID(FK)**, **Booking_ID(FK)**)

Relation 7: Can Be Made (Booking Can Be Made On Turf)

UNF:

Can_Be_Made(Booking_ID, Start_Time, End_Time, Date, Status, Turf_ID, Turf_Name, Location, Size, Type)

1NF:

No multivalued attributes, so it remains

1. Booking_ID, Start_Time, End_Time, Date, Status, Turf_ID, Turf_Name, Location, Size, Type

2NF:

Booking details depend on Booking_ID

Turf details depend on Turf_ID

1. Booking_ID, Start_Time, End_Time, Date, Status

2. Turf_ID, Turf_Name, Location, Size, Type

3. Booking_ID, Turf_ID

3NF:

No Transitive dependencies.

1. Booking_ID, Start_Time, End_Time, Date, Status

2. Turf_ID, Turf_Name, Location, Size, Type

3. Booking_ID, Turf_ID

Table Creation:

Booking(**Booking_ID(PK)**, Start_Time, End_Time, Date, Status)

Turf(**Turf_ID(PK)**, Turf_Name, Location, Size, Type)

Can_Be_Made(**Booking_ID(FK)**, **Turf_ID(FK)**)

Relation 8: Can Have (Booking Can Have Payment)

UNF:

Can_Have(Booking_ID, Start_Time, End_Time, Date, Status, Payment_ID, Amount, Payment_Date)

1NF:

No multivalued attributes, so it remains

1. Booking_ID, Start_Time, End_Time, Date, Status, Payment_ID, Amount, Payment_Date

2NF:

Booking details depend on Booking_ID

Payment details depend on Payment_ID

1. Booking(Booking_ID, Start_Time, End_Time, Date, Status)

2. Payment(Payment_ID, Amount, Payment_Date)

3. Can_Have(Booking_ID, Payment_ID)

3NF:

No Transitive dependencies.

1. Booking(Booking_ID, Start_Time, End_Time, Date, Status)

2. Payment(Payment_ID, Amount, Payment_Date)

3. Can_Have(Booking_ID, Payment_ID)

Table Creation:

Booking(**Booking_ID(PK)**, Start_Time, End_Time, Date, Status)

Payment(**Payment_ID(PK)**, Amount, Payment_Date)

Can_Have(**Booking_ID(FK)**, **Payment_ID(FK)**)

Relation 9: Can Use (Payment Can use Payment Type)

UNF:

Can_Use(Payment_ID, Amount, Payment_Date, Payment_Type_Code, Payment_Method, Description)

1NF:

No multivalued attributes, so it remains

1. Payment_ID, Amount, Payment_Date, Payment_Type_Code, Payment_Method, Description

2NF:

Payment details depend on Payment_ID

Payment type details depend on Payment_Type_Code

1. Payment(Payment_ID, Amount, Payment_Date, Payment_Type_Code)

2. Payment_Type(Payment_Type_Code, Payment_Method, Description

3NF:

No Transitive dependencies.

1. Payment(Payment_ID, Amount, Payment_Date, Payment_Type_Code)

2. Payment_Type(Payment_Type_Code, Payment_Method, Description

Table Creation:

Payment(**Payment_ID(PK)**, Amount, Payment_Date, **Payment_Type_Code(FK)**)

Payment_Type(**Payment_Type_Code(PK)**, Payment_Method, Description)

Final Table:

Entity Tables

1. **Staff_Member:** (**Staff_ID(PK)**, First_Name, Last_Name, Department)
2. **Event:** (**Event_ID(PK)**, Event_Name, Date, Rules)
3. **Team:** (**Team_ID(PK)**, Team_Name, Sport_Type)
4. **Player:** (**Player_ID(PK)**, First_Name, Last_Name, DOB, Phone_Number)
5. **Profile:** (**Profile_ID(PK)**, Skill_Level, Achievements)
6. **Booking:** (**Booking_ID(PK)**, Start_Time, End_Time, Date, Status)
7. **Turf:** (**Turf_ID(PK)**, Turf_Name, Location, Size, Type)
8. **Payment:** (**Payment_ID(PK)**, Amount, Payment_Date, **Payment_Type_Code(FK)**)
9. **Payment_Type:** (**Payment_Type_Code(PK)**, Payment_Method, Description)

Relationship Tables

10. **Can_Manage:** (**Staff_ID(FK), Event_ID(FK)**)
11. **Participates:** (**Event_ID(FK), Team_ID(FK)**)
12. **Belongs_To:** (**Player_ID(FK), Team_ID(FK)**)
13. **Has_Profile:** (**Player_ID(FK), Profile_ID(FK)**)
14. **Can_Make:** (**Team_ID(FK), Booking_ID(FK)**)
15. **Makes:** (**Player_ID(FK), Booking_ID(FK)**)
16. **Can_Be_Made:** (**Booking_ID(FK), Turf_ID(FK)**)
17. **Can_Have:** (**Booking_ID(FK), Payment_ID(FK)**)
18. **Can_Use:** (**Payment_ID(FK), Payment_Type_Code(FK)**)

2. SCHEMA DIAGRAM

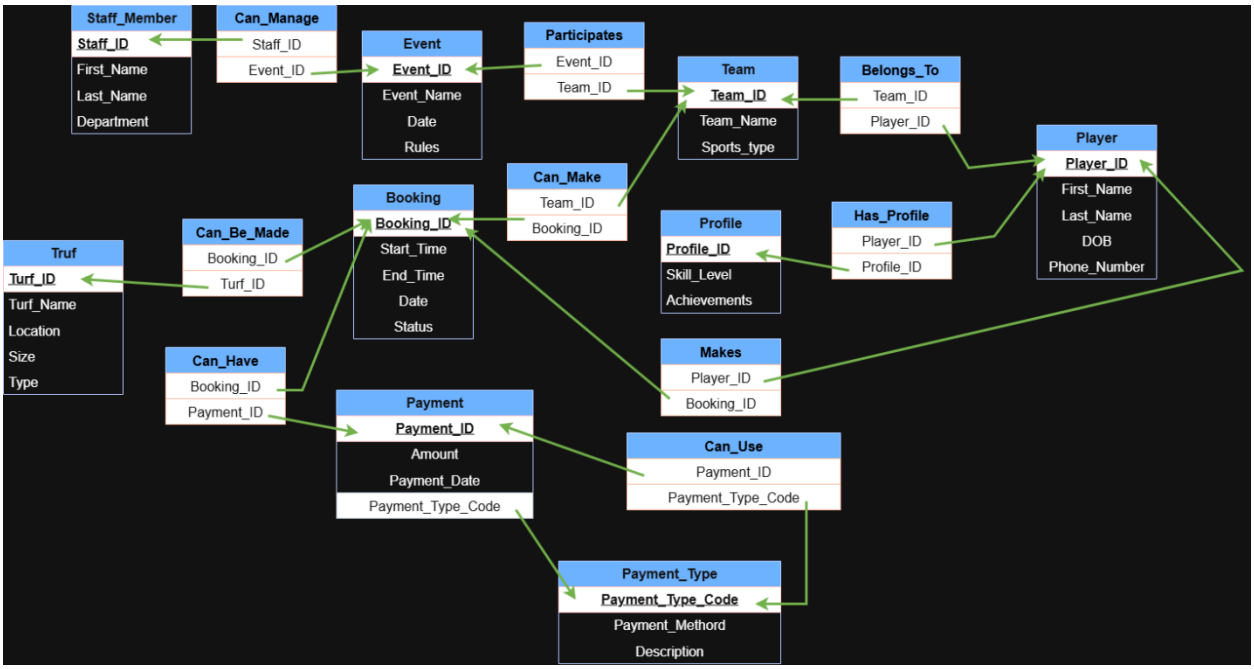
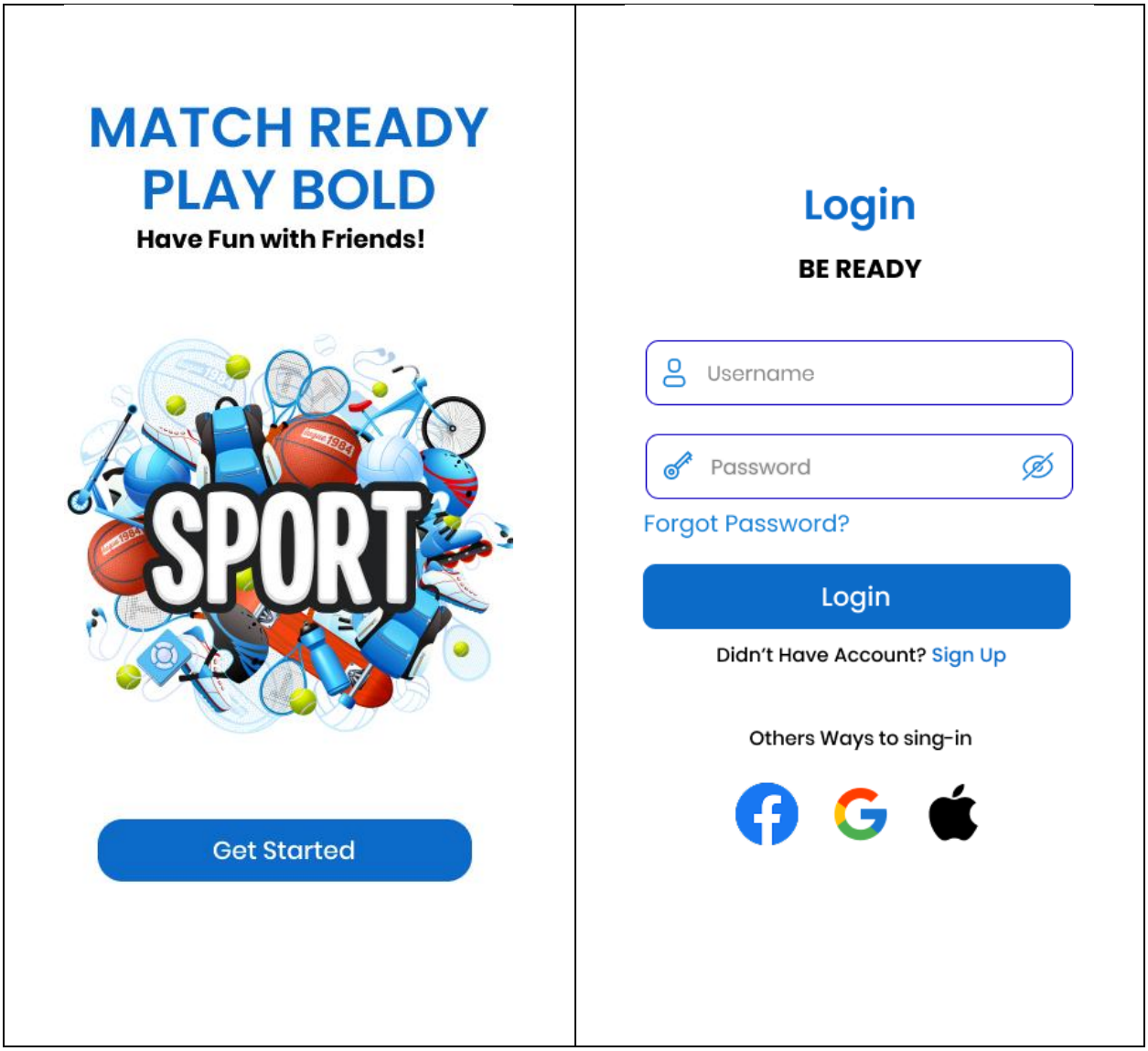


Fig-1: SCHEMA DIAGRAM

3. USER INTERFACE: (Figma)

Figma Live link:

https://www.figma.com/design/gu6MmmA9LLIFNvthsJOYzK/Outdoor-sports-management?node-id=0-1&p=f&t=R19DcoDAKIZGseXV-0



Sign - Up

Username

Email Id

Contact Number

New Password

Confirm Password

Sign-Up

Didn't Have Account? Sign Up

Others Ways to sing-in

Forgot Password

Verify with OTP to change Password sent to 0181**938**

Auto fetching OTP...

Submit

Didn't receive it? Retry in 00:60

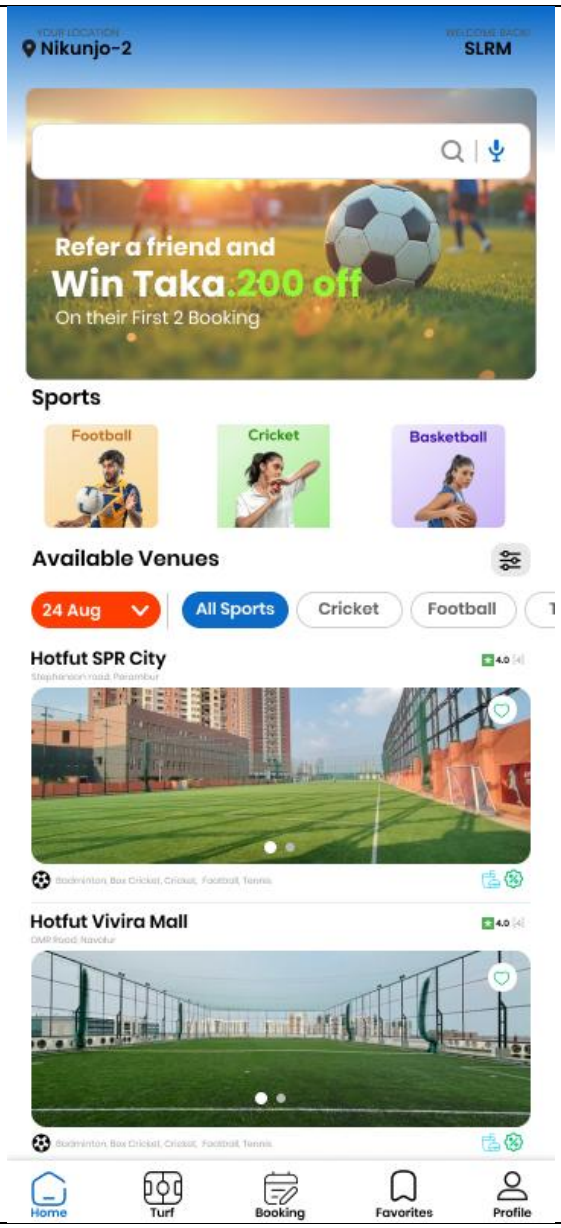
Save Password

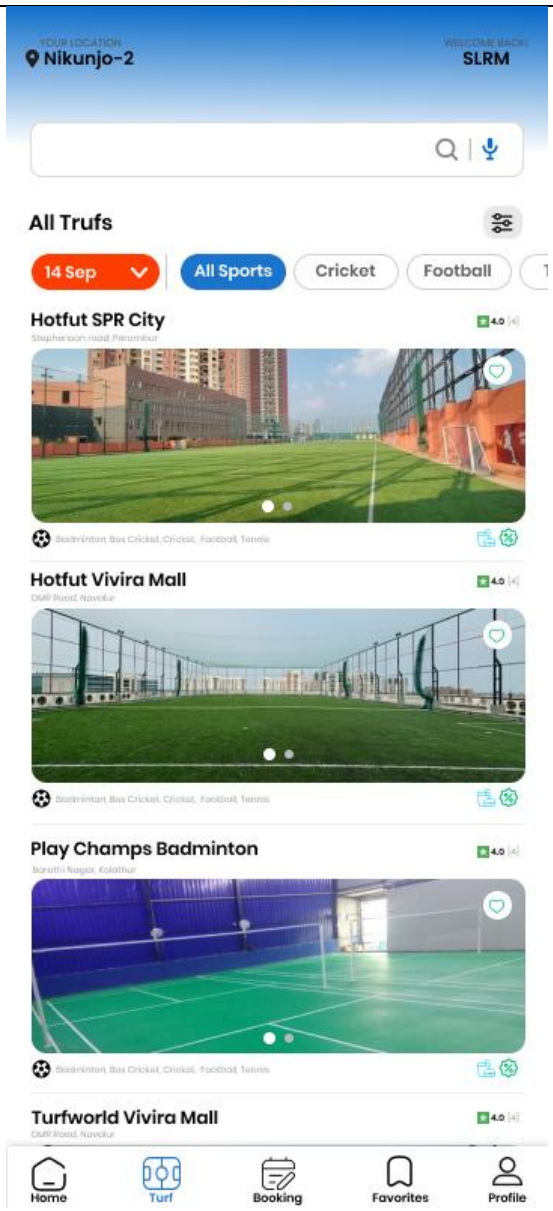
Verify with OTP to change Password sent to 0181**938**

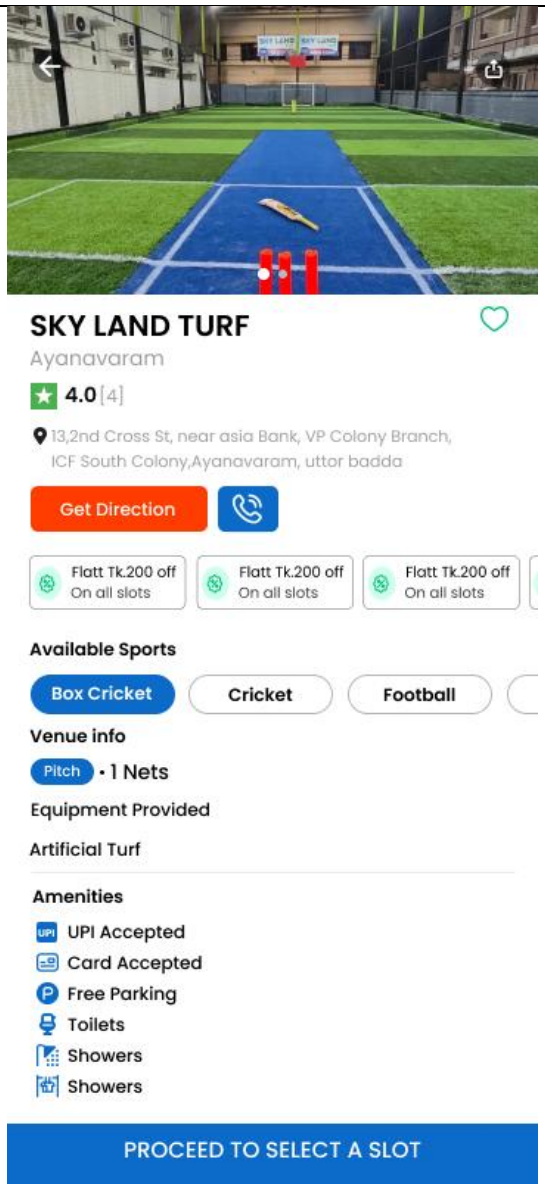
New Password

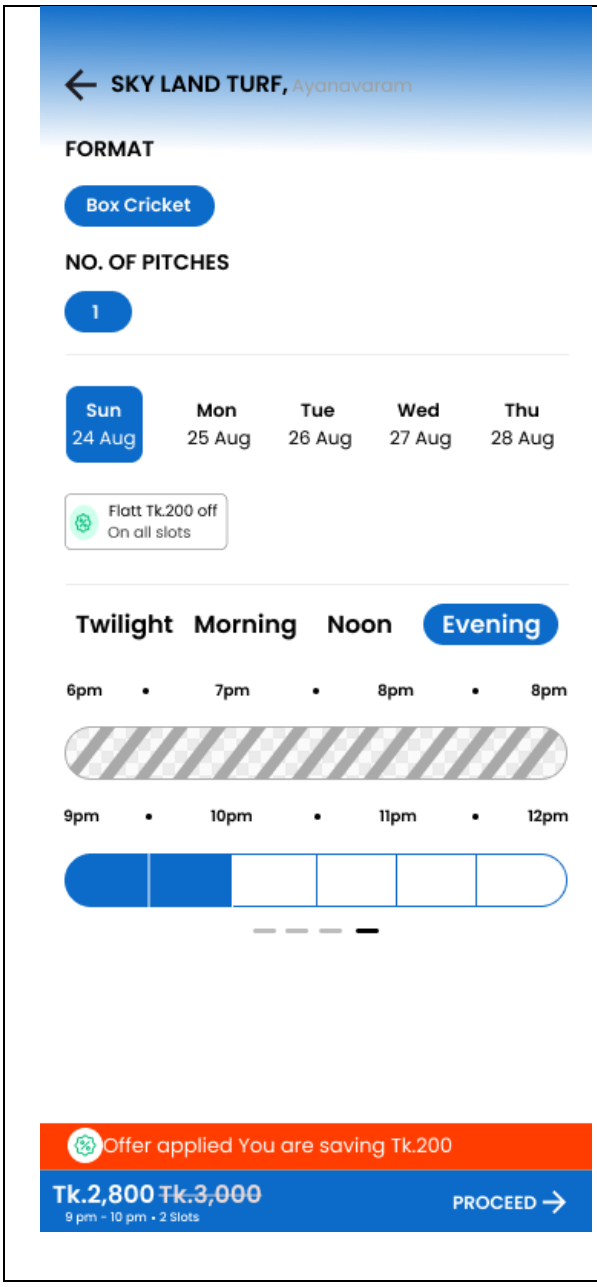
Confirm Password

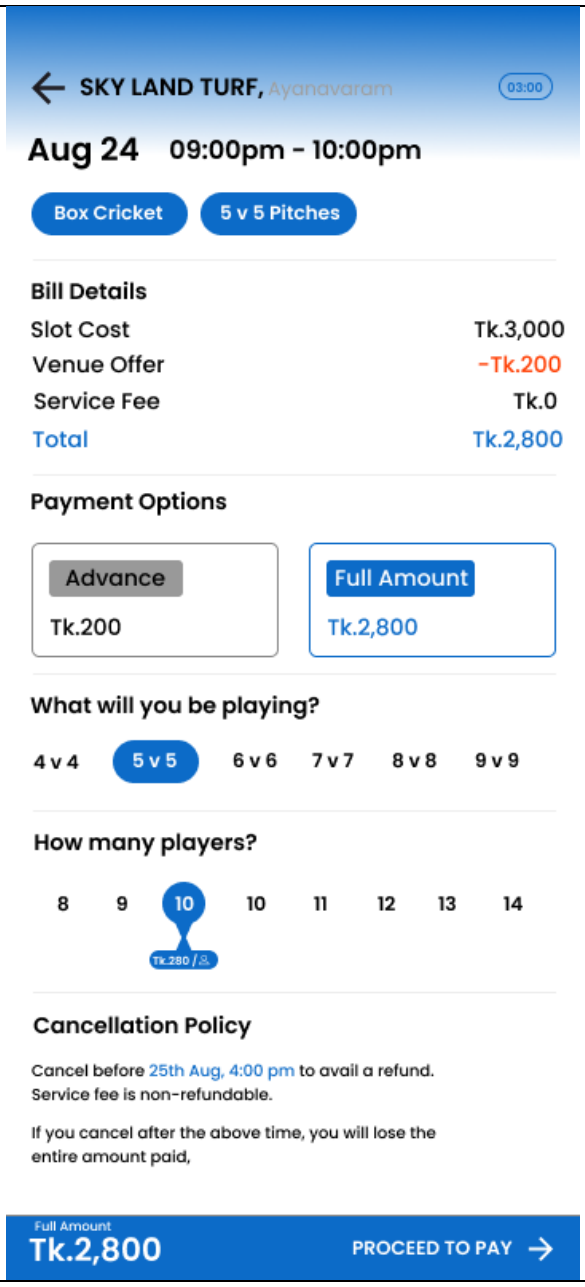
Save Password

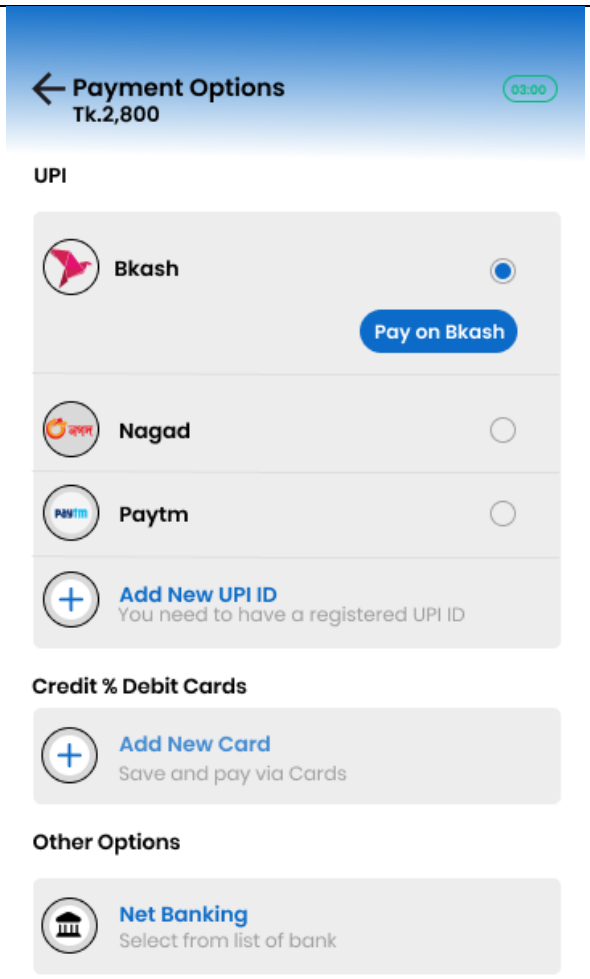


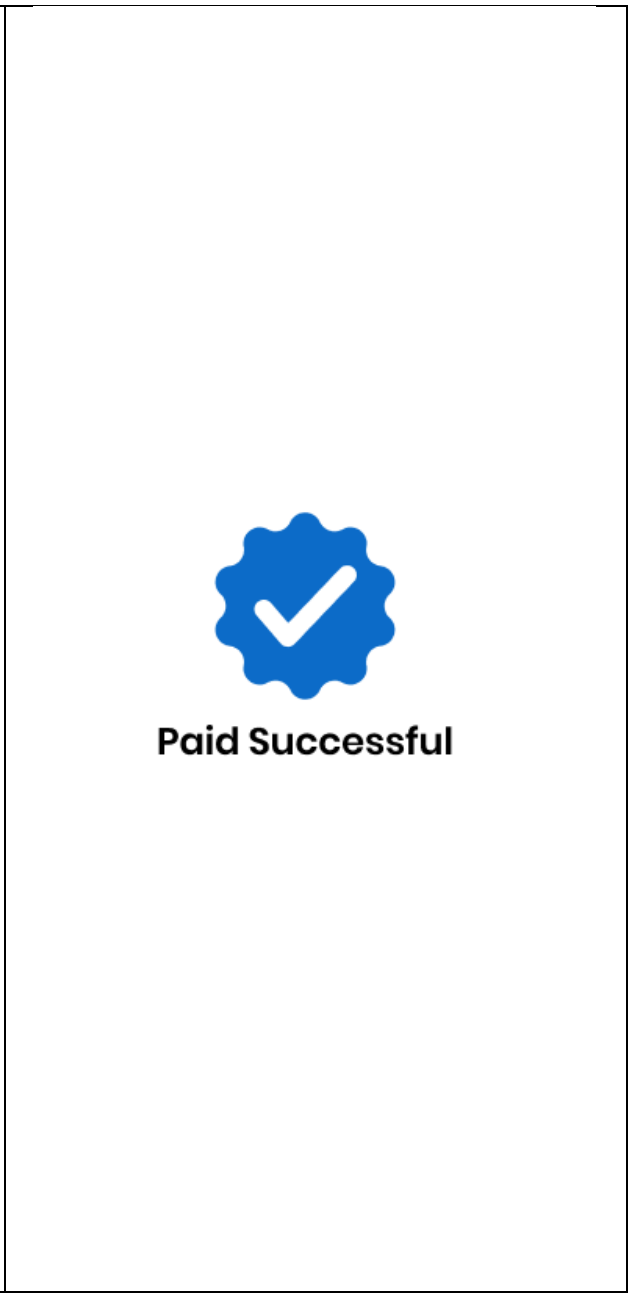

YOUR LOCATION
Nikunjo-2
WELCOME BACK!
SLRM
All Trufs
14 Sep
All Sports
Cricket
Football
Hotfut SPR City
4.0 [4]
Hotfut Vivira Mall
4.0 [4]
Play Champs Badminton
4.0 [4]
Turfworld Vivira Mall
4.0 [4]
Home
Turf
Booking
Favorites
Profile


SKY LAND TURF
Ayanavaram
4.0 [4]
13,2nd Cross St, near asia Bank, VP Colony Branch, ICF South Colony,Ayanavaram, uttor badda
Get Direction
Flatt Tk.200 off On all slots
Flatt Tk.200 off On all slots
Flatt Tk.200 off On all slots
Available Sports
Box Cricket
Cricket
Football
Venue info
Pitch
• 1 Nets
Equipment Provided
Artificial Turf
Amenities
UPI Accepted
Card Accepted
Free Parking
Toilets
Showers
Showers
PROCEED TO SELECT A SLOT

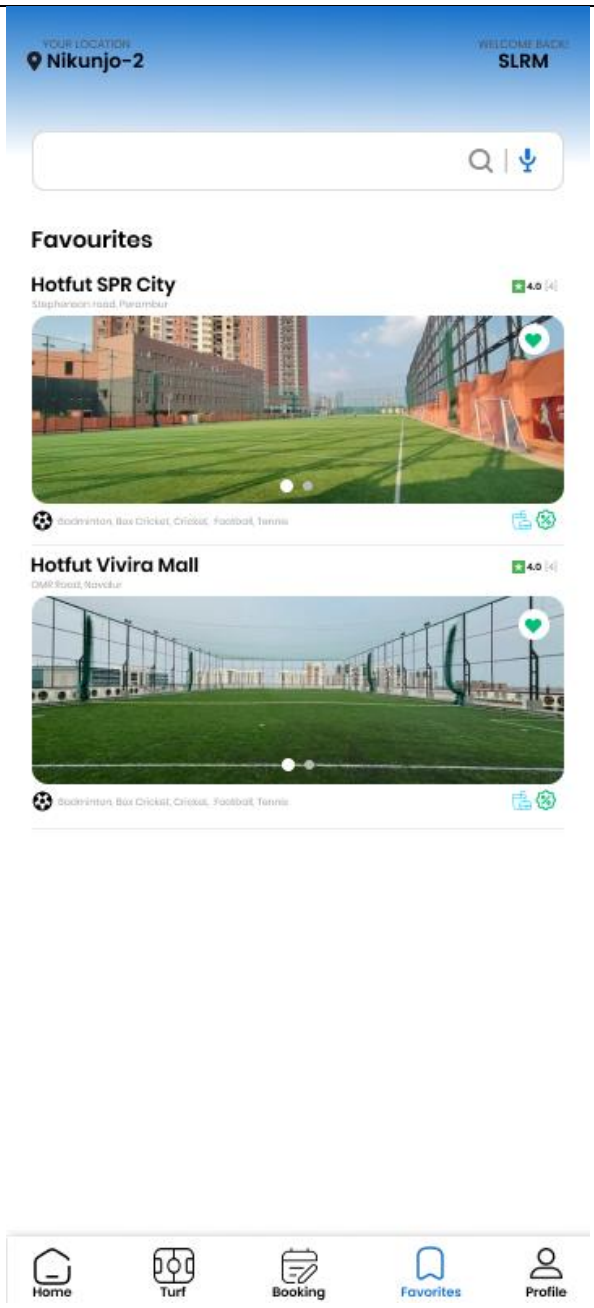

SKY LAND TURF, Ayanavaram
FORMAT
Box Cricket
NO. OF PITCHES
1
Sun 24 Aug
Mon 25 Aug
Tue 26 Aug
Wed 27 Aug
Thu 28 Aug
Flatt Tk.200 off
On all slots
Twilight
Morning
Noon
Evening
6pm • 7pm • 8pm • 8pm
9pm • 10pm • 11pm • 12pm
Offer applied You are saving Tk.200
Tk.2,800 Tk.3,000
9 pm - 10 pm • 2 Slots
PROCEED

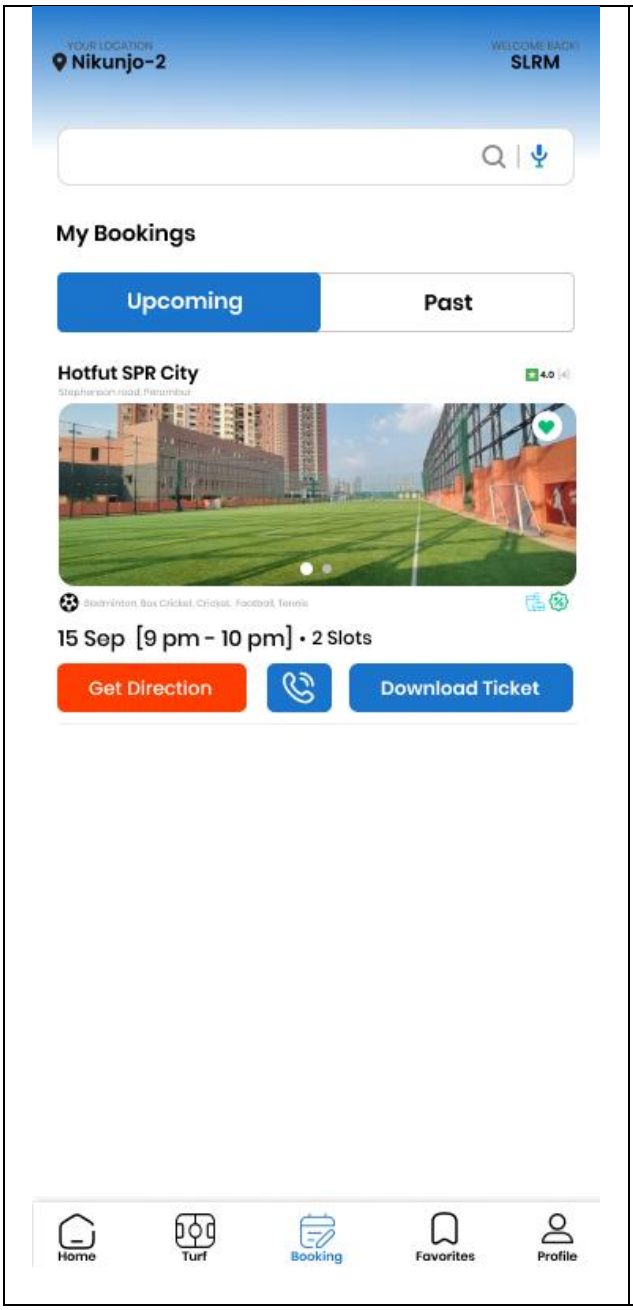

SKY LAND TURF, Ayanavaram
03:00
Aug 24 09:00pm - 10:00pm
Box Cricket
5 v 5 Pitches
Bill Details
Slot Cost Tk.3,000
Venue Offer -Tk.200
Service Fee Tk.0
Total Tk.2,800
Payment Options
Advance
Tk.200
Full Amount
Tk.2,800
What will you be playing?
4 v 4
5 v 5
6 v 6
7 v 7
8 v 8
9 v 9
How many players?
8 9 10 10 11 12 13 14
Tk.280 /
Cancellation Policy
Cancel before 25th Aug, 4:00 pm to avail a refund.
Service fee is non-refundable.
If you cancel after the above time, you will lose the entire amount paid,
Full Amount
Tk.2,800
PROCEED TO PAY

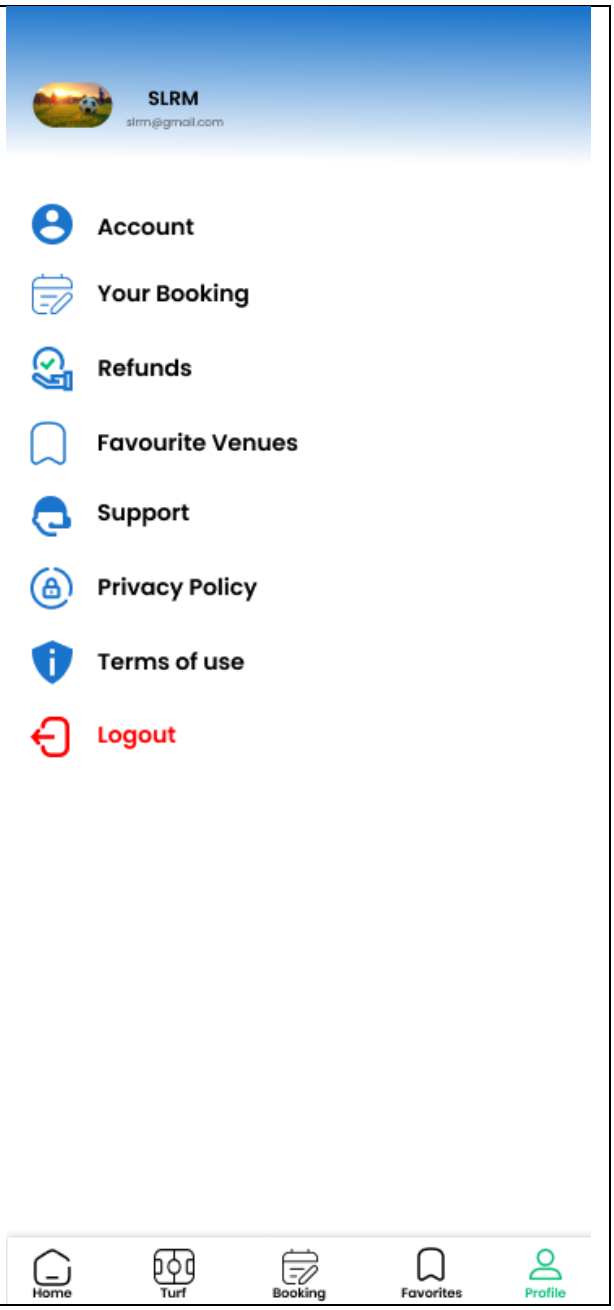

Payment Options
Tk.2,800
03:00
UPI
Bkash
Pay on Bkash
Nagad
Paytm
Add New UPI ID
You need to have a registered UPI ID
Credit % Debit Cards
Add New Card
Save and pay via Cards
Other Options
Net Banking
Select from list of bank


Paid Successful


Congratulations!
Your Slot has been booked!
TICKET
Scan this QR on the Scanner Machine When you are in Entrence
NAME
slmr
ID
Surendhar_PV
MOBILE
018166049**
Sport Name
Box Cricket
Turf Ground Name
Sky Land Truf
Address
Badda
Booking Time
9:00-10:00 PM
Date
24-08-2025
Players
5 v 5
Amount
Tk. 2800
Transaction ID
21487987329578
Home
Download Ticket


Nikunjo-2
SLRM
Favourites
Hotfut SPR City
Hotfut Vivira Mall
Home
Turf
Booking
Favorites
Profile


YOUR LOCATION
Nikunjo-2
WELCOME BACK!
SLRM
My Bookings
Upcoming
Past
Hotfut SPR City
15 Sep [9 pm - 10 pm] • 2 Slots
Get Direction
Download Ticket
Home
Turf
Booking
Favorites
Profile


SLRM
slrm@gmail.com
Account
Your Booking
Refunds
Favourite Venues
Support
Privacy Policy
Terms of use
Logout
Home
Turf
Booking
Favorites
Profile

4. **Advance PL/SQL:**

    1. **Stored function**

Question-1: Create a function to retrieve the skill level of a player based on their Player_ID.

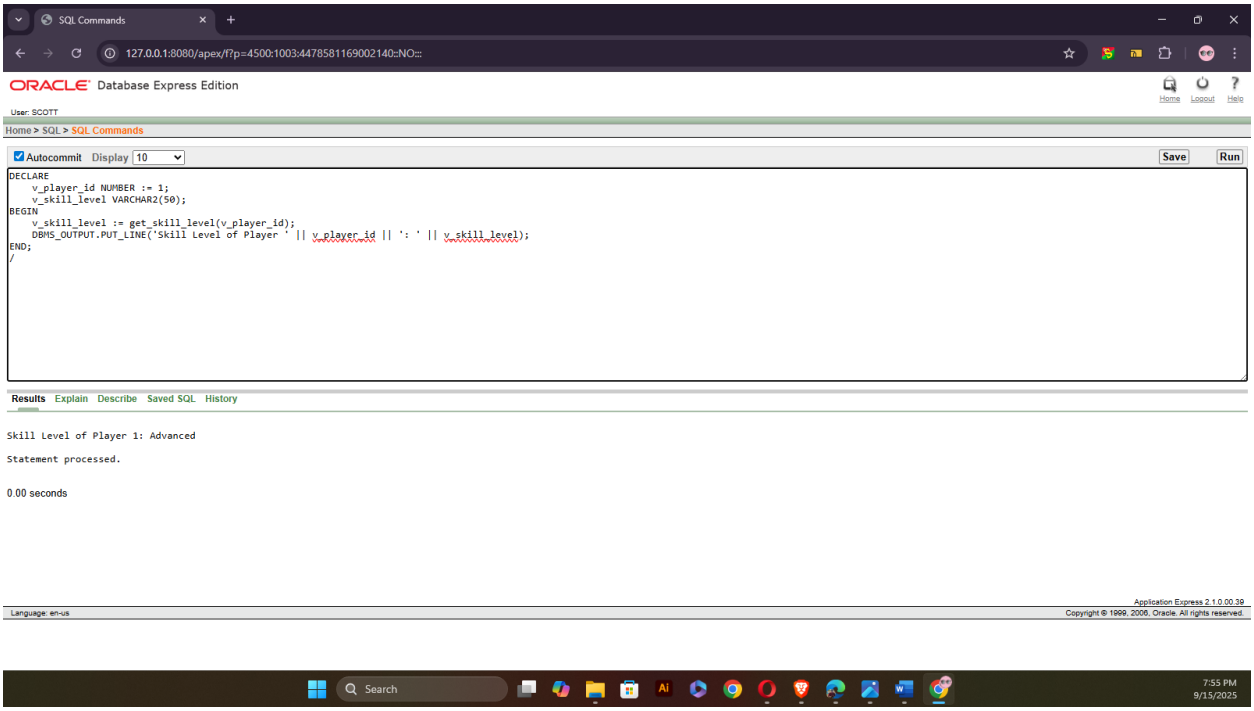
Answer:

```
-- Project: Outdoor Sports Management System
-- Semester: Summer 2025
-- Course Name: Advance Database Management Systems
-- Section: A


CREATE OR REPLACE FUNCTION get_skill_level (p_player_id IN NUMBER)
RETURN VARCHAR2 IS
  v_skill_level VARCHAR2(50);
BEGIN

  SELECT pr.Skill_Level
  INTO v_skill_level
  FROM Profile pr
  JOIN Has_Profile hp ON pr.Profile_ID = hp.Profile_ID
  WHERE hp.Player_ID = p_player_id;

  RETURN v_skill_level;

EXCEPTION
  WHEN NO_DATA_FOUND THEN
    RETURN 'No skill level found for this player';
  WHEN OTHERS THEN
```

```
    RETURN 'Error retrieving skill level';
END get_skill_level;
/
```

To see the answer

```
-- Project: Outdoor Sports Management System
-- Semester: Summer 2025
-- Course Name: Advance Database Management Systems
-- Section: A

DECLARE
  v_player_id NUMBER := 1;
  v_skill_level VARCHAR2(50);
BEGIN
  v_skill_level := get_skill_level(v_player_id);
  DBMS_OUTPUT.PUT_LINE('Skill Level of Player ' || v_player_id || ': ' || v_skill_level);
END;
/
```



Question-2: Create a function to retrieve the turf location based on the Booking_ID.

Answer:

-- Project: Outdoor Sports Management System

-- Semester: Summer 2025

-- Course Name: Advance Database Management Systems

-- Section: A

```
CREATE OR REPLACE FUNCTION get_turf_location (p_booking_id IN NUMBER)
RETURN VARCHAR2 IS
  v_turf_location VARCHAR2(100);
BEGIN

  SELECT t.Turf_location
  INTO v_turf_location
  FROM Turf t
```

```
  JOIN Can_Be_Made cbm ON t.Turf_ID = cbm.Turf_ID
  WHERE cbm.Booking_ID = p_booking_id;

  RETURN v_turf_location;

EXCEPTION
  WHEN NO_DATA_FOUND THEN
    RETURN 'No turf location found for this booking';
  WHEN OTHERS THEN
    RETURN 'Error retrieving turf location';
END get_turf_location;
/
```

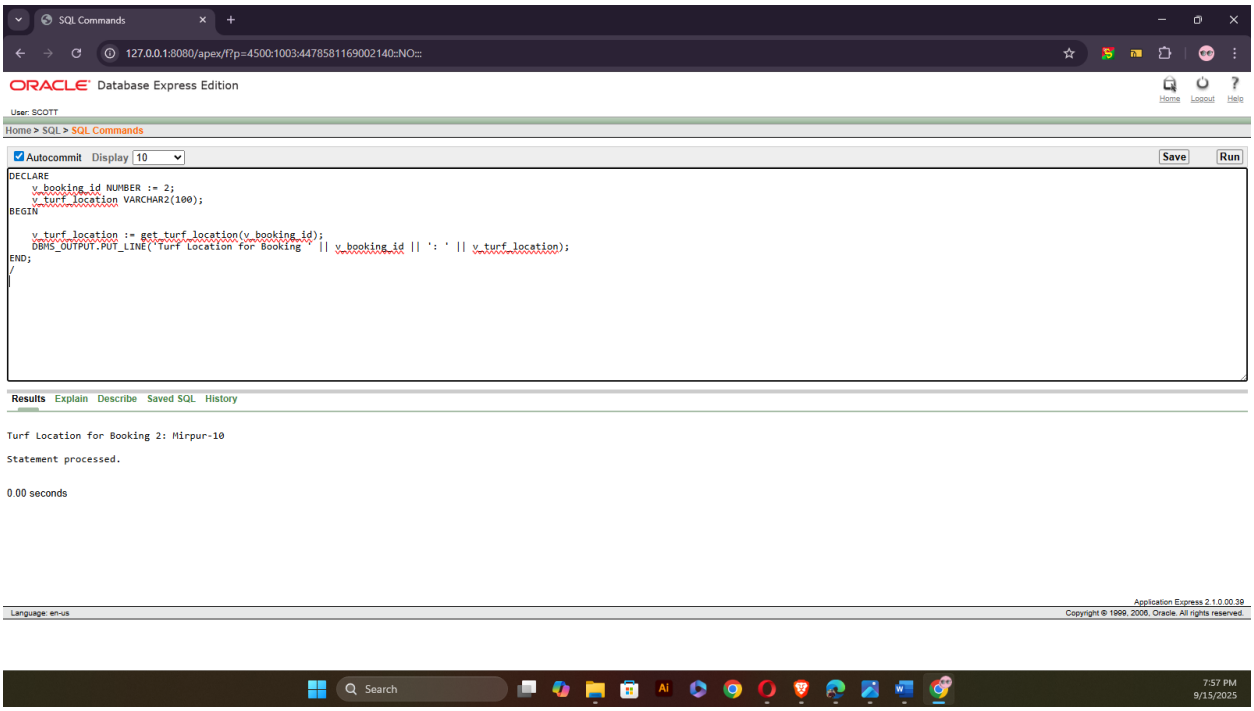
To see the answer:

```
-- to see the function
-- Project: Outdoor Sports Management System
-- Semester: Summer 2025
-- Course Name: Advance Database Management Systems
-- Section: A

DECLARE
  v_booking_id NUMBER := 2;
  v_turf_location VARCHAR2(100);
BEGIN

  v_turf_location := get_turf_location(v_booking_id);
```

DBMS_OUTPUT.PUT_LINE('Turf Location for Booking ' || v_booking_id || ': ' || v_turf_location);

END;

/



2. Stored procedure

Question-1: Create a stored procedure to update the skill level of a player based on their Player_ID

Answer:

-- Project: Outdoor Sports Management System

-- Semester: Summer 2025

-- Course Name: Advance Database Management Systems

-- Section: A

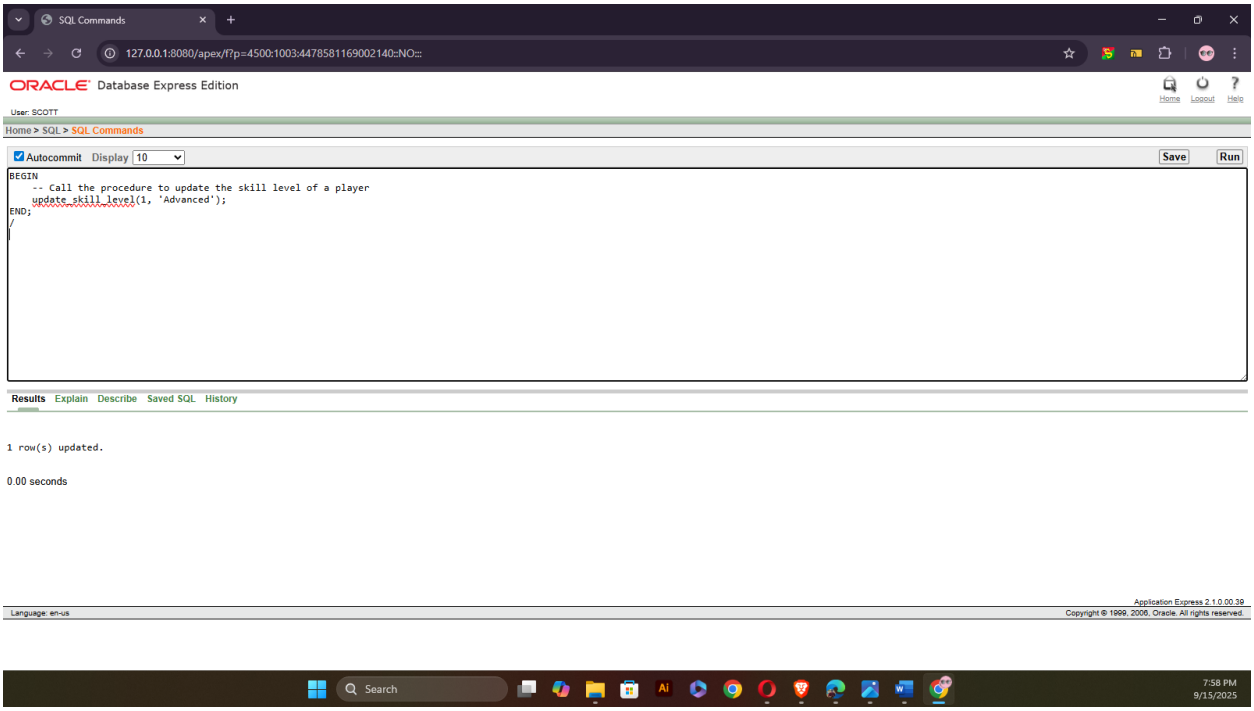
CREATE OR REPLACE PROCEDURE update_skill_level (p_player_id IN NUMBER, p_skill_level IN VARCHAR2)

```
IS
BEGIN

    UPDATE Profile
    SET Skill_Level = p_skill_level
    WHERE Profile_ID = (SELECT Profile_ID FROM Has_Profile WHERE Player_ID =
p_player_id);

    IF SQL%ROWCOUNT = 0 THEN
        RAISE_APPLICATION_ERROR(-20001, 'No player found with the given Player_ID');
    END IF;

    COMMIT;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No matching player found.');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error updating skill level: ' || SQLERRM);
END update_skill_level;
/
```

To test the procedure:

```
-- to see the function
-- Project: Outdoor Sports Management System
-- Semester: Summer 2025
-- Course Name: Advance Database Management Systems
-- Section: A
```

```
BEGIN
    -- Call the procedure to update the skill level of a player
    update_skill_level(1, 'Advanced');
END;
/
```



Question-2: Create a stored procedure to update the booking status of a specific booking based on the Booking_ID.

Answer:

-- Project: Outdoor Sports Management System

-- Semester: Summer 2025

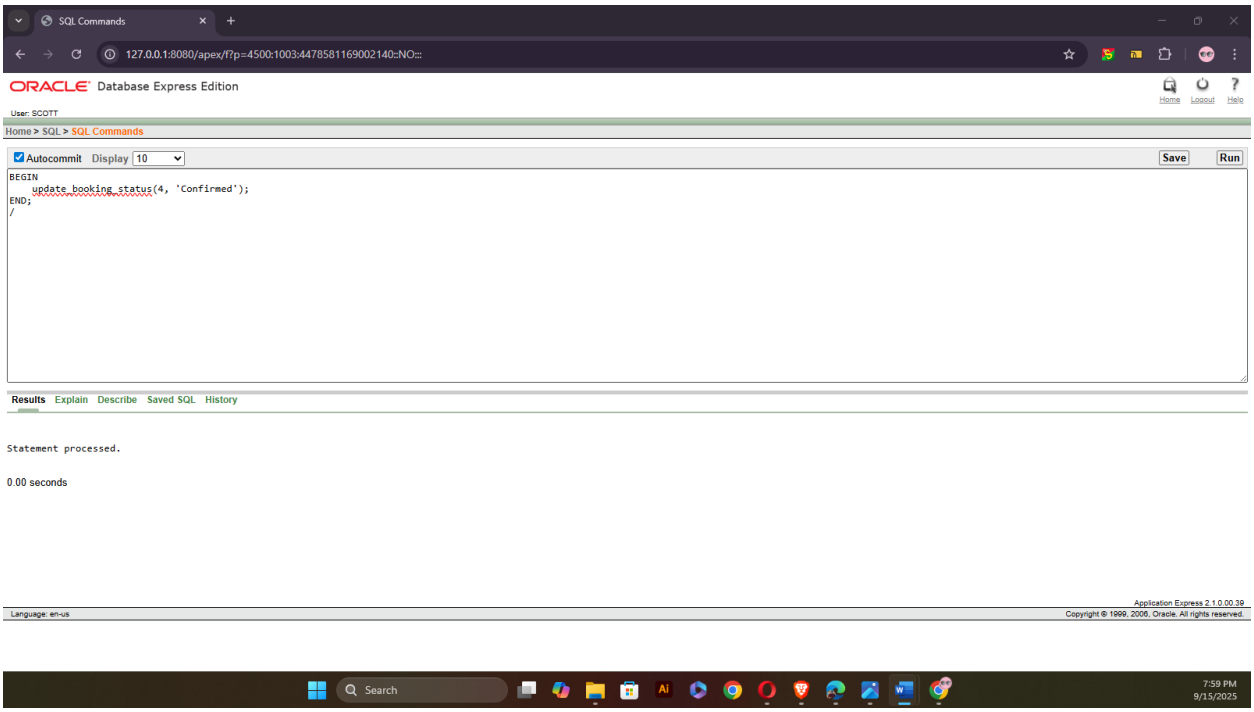
-- Course Name: Advance Database Management Systems

-- Section: A

CREATE OR REPLACE PROCEDURE update_booking_status (p_booking_id IN NUMBER, p_status IN VARCHAR2)

```
IS
BEGIN

  UPDATE Booking
  SET Status = p_status
  WHERE Booking_ID = p_booking_id;

  IF SQL%ROWCOUNT = 0 THEN
    RAISE_APPLICATION_ERROR(-20002, 'No booking found with the given Booking_ID');
  END IF;

  COMMIT;

EXCEPTION
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('No booking found.');
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Error updating booking status: ' || SQLERRM);
END update_booking_status;
/
To test the procedure:
-- Project: Outdoor Sports Management System
-- Semester: Summer 2025
-- Course Name: Advance Database Management Systems
-- Section: A
BEGIN
```

```
    update_booking_status(4, 'Confirmed');
END;
/
```



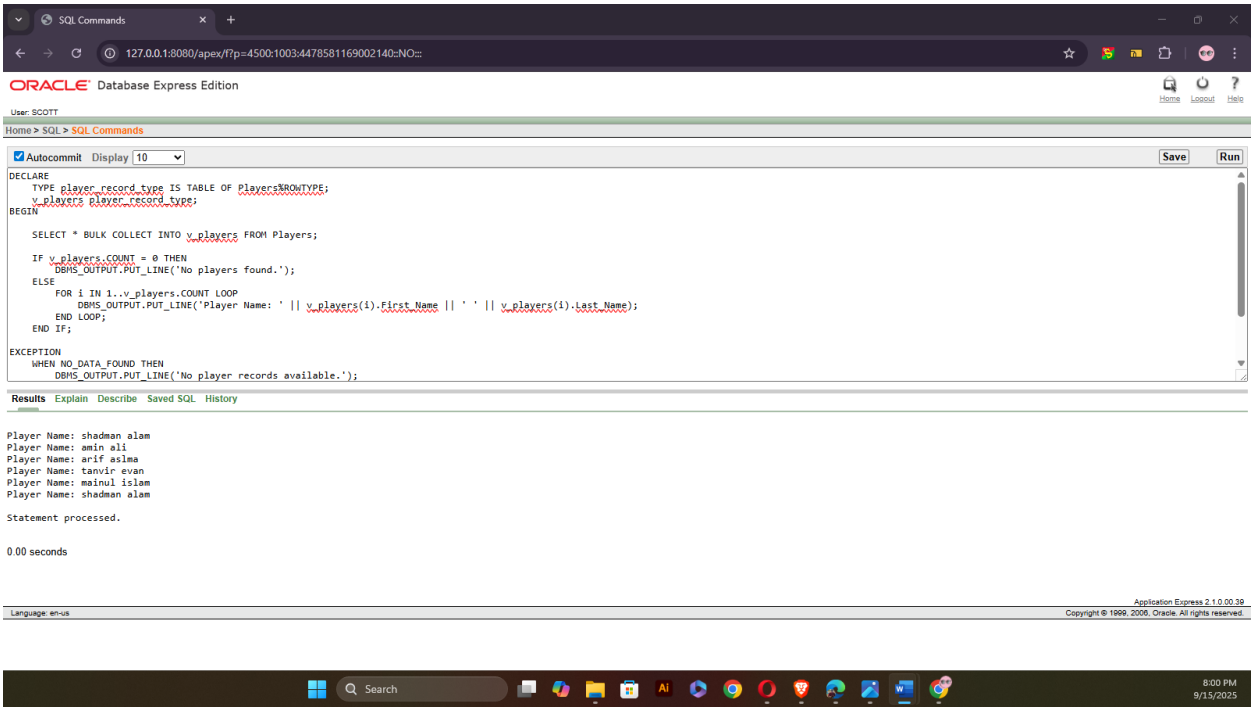
3. Table record

Question-1: Create a table-based record type to store player data and loop through it.

Answer:

```
-- Project: Outdoor Sports Management System
-- Semester: Summer 2025
-- Course Name: Advance Database Management Systems
-- Section: A


DECLARE
    TYPE player_record_type IS TABLE OF Players%ROWTYPE;
    v_players player_record_type;
BEGIN
```

```
    SELECT * BULK COLLECT INTO v_players FROM Players;


    IF v_players.COUNT = 0 THEN
        DBMS_OUTPUT.PUT_LINE('No players found.');
    ELSE
        FOR i IN 1..v_players.COUNT LOOP
            DBMS_OUTPUT.PUT_LINE('Player Name: ' || v_players(i).First_Name || ' ' ||
v_players(i).Last_Name);
        END LOOP;
    END IF;


EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No player records available.');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error retrieving player records: ' || SQLERRM);
END;
/
```



Question-2: Create a table-based record type to store booking data and loop through it.

Answer:

-- Project: Outdoor Sports Management System

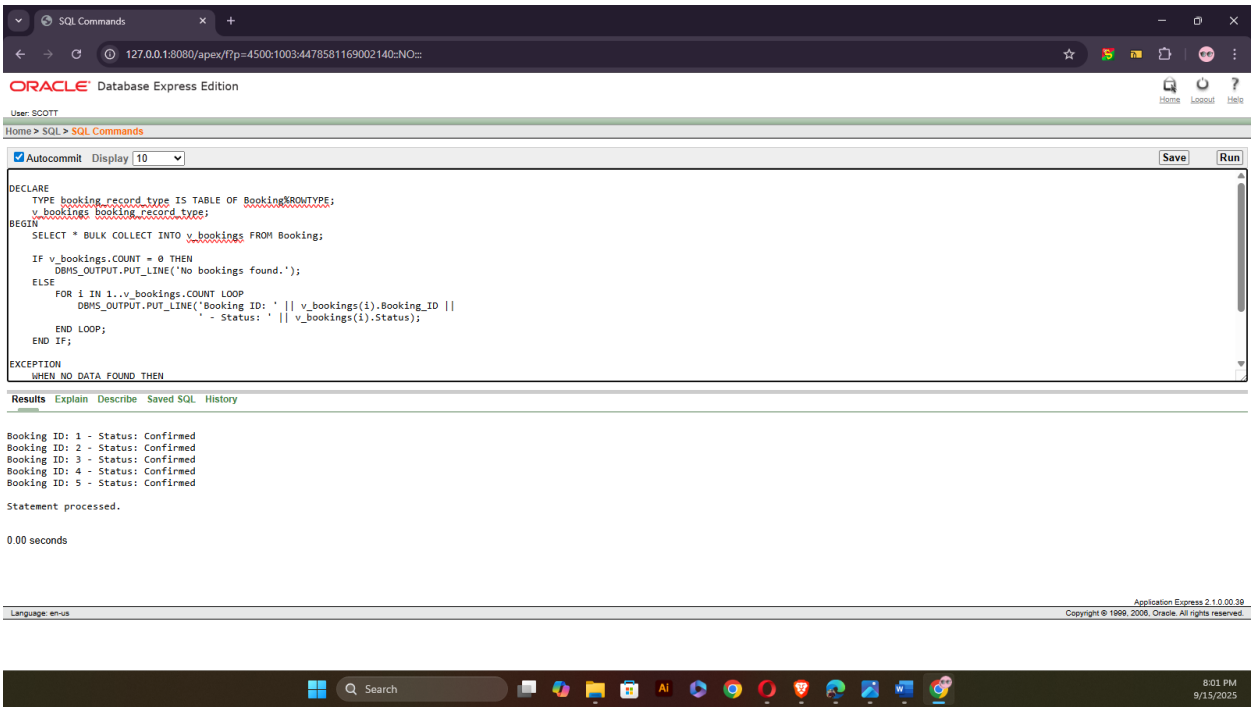
-- Semester: Summer 2025

-- Course Name: Advance Database Management Systems

-- Section: A

```
DECLARE
  TYPE booking_record_type IS TABLE OF Booking%ROWTYPE;
  v_bookings booking_record_type;
BEGIN
  SELECT * BULK COLLECT INTO v_bookings FROM Booking;

  IF v_bookings.COUNT = 0 THEN
    DBMS_OUTPUT.PUT_LINE('No bookings found.');
```

```
  ELSE
    FOR i IN 1..v_bookings.COUNT LOOP
      DBMS_OUTPUT.PUT_LINE('Booking ID: ' || v_bookings(i).Booking_ID ||
                           ' - Status: ' || v_bookings(i).Status);
    END LOOP;
  END IF;

EXCEPTION
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('No booking records available.');
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Error retrieving booking records: ' || SQLERRM);
END;
/
```



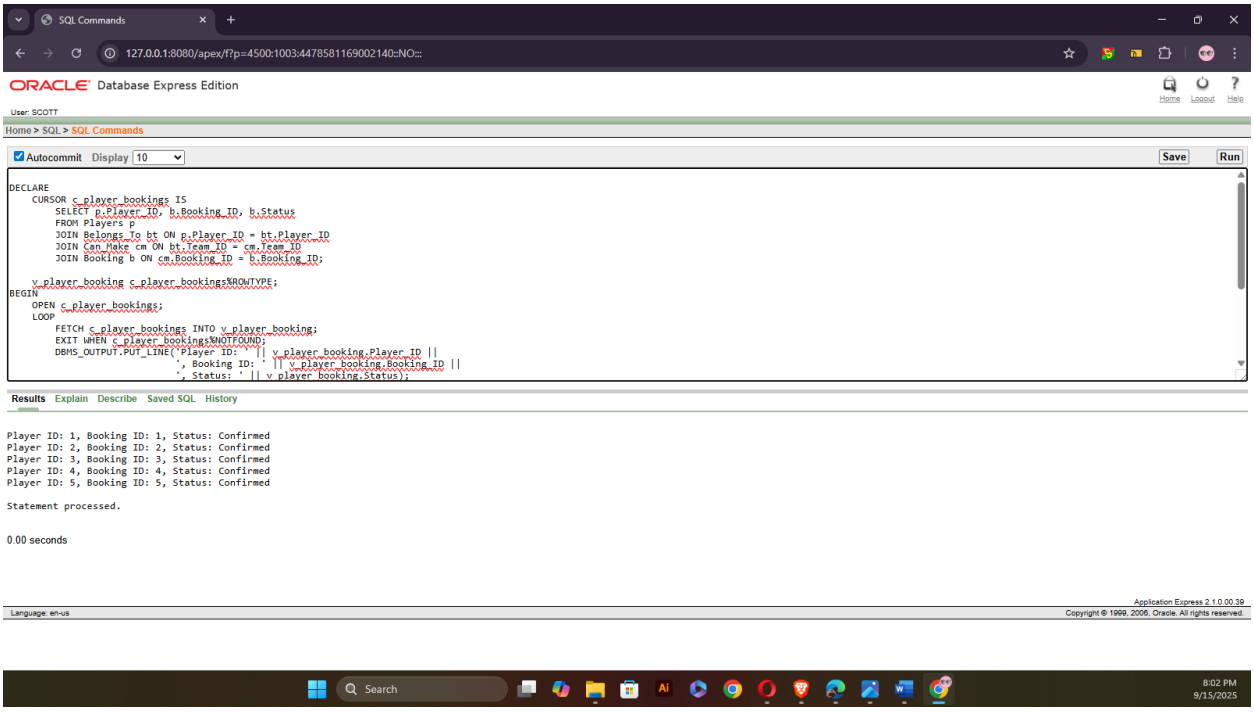
4. Explicit cursor

Question-1: Create an explicit cursor to fetch players with their booking details.

Answer:

```
-- Project: Outdoor Sports Management System
-- Semester: Summer 2025
-- Course Name: Advance Database Management Systems
-- Section: A

DECLARE
   CURSOR c_player_bookings IS
      SELECT p.Player_ID, b.Booking_ID, b.Status
      FROM Players p
      JOIN Belongs_To bt ON p.Player_ID = bt.Player_ID
      JOIN Can_Make cm ON bt.Team_ID = cm.Team_ID
      JOIN Booking b ON cm.Booking_ID = b.Booking_ID;

   v_player_booking c_player_bookings%ROWTYPE;
BEGIN
   OPEN c_player_bookings;
   LOOP
      FETCH c_player_bookings INTO v_player_booking;
      EXIT WHEN c_player_bookings%NOTFOUND;
      DBMS_OUTPUT.PUT_LINE('Player ID: ' || v_player_booking.Player_ID ||
                           ', Booking ID: ' || v_player_booking.Booking_ID ||
                           ', Status: ' || v_player_booking.Status);
   END LOOP;
   CLOSE c_player_bookings;
```

```
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No player booking records found.');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error fetching player bookings: ' || SQLERRM);
END;
/
```



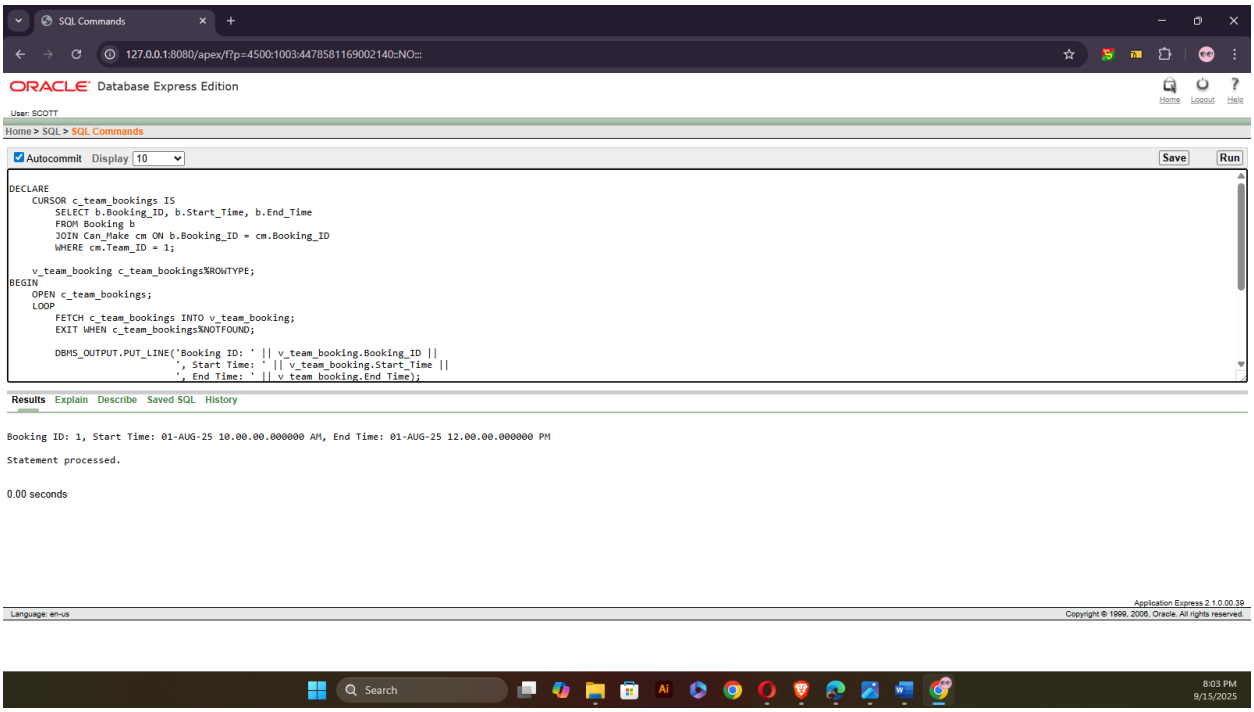
Question-2: Create an explicit cursor to fetch the booking details (Booking ID, Start Time, and End Time) for a specific team.

Answer:

```
-- Project: Outdoor Sports Management System
-- Semester: Summer 2025
-- Course Name: Advance Database Management Systems
-- Section: A
```

```
DECLARE
  CURSOR c_team_bookings IS
    SELECT b.Booking_ID, b.Start_Time, b.End_Time
    FROM Booking b
    JOIN Can_Make cm ON b.Booking_ID = cm.Booking_ID
    WHERE cm.Team_ID = 1;

  v_team_booking c_team_bookings%ROWTYPE;
BEGIN
  OPEN c_team_bookings;
  LOOP
    FETCH c_team_bookings INTO v_team_booking;
    EXIT WHEN c_team_bookings%NOTFOUND;

    DBMS_OUTPUT.PUT_LINE('Booking ID: ' || v_team_booking.Booking_ID ||
                         ', Start Time: ' || v_team_booking.Start_Time ||
                         ', End Time: ' || v_team_booking.End_Time);
  END LOOP;
  CLOSE c_team_bookings;

EXCEPTION
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('No booking records found for the given team.');
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Error fetching team booking details: ' || SQLERRM);
END;
```

/



5. Cursor-based-record

Question-1: Use a cursor-based record to print the names of players who have confirmed bookings.

Answer:

-- Project: Outdoor Sports Management System

-- Semester: Summer 2025

-- Course Name: Advance Database Management Systems

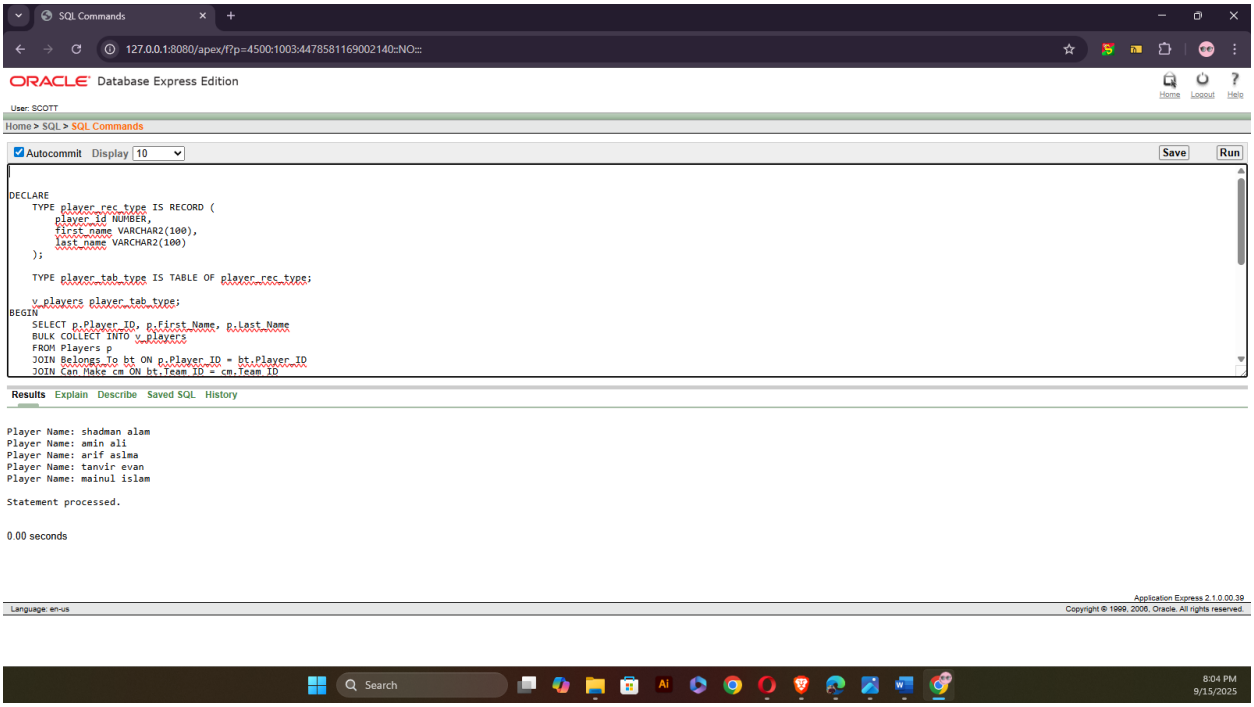
-- Section: A

```
DECLARE
   TYPE player_rec_type IS RECORD (
      player_id NUMBER,
      first_name VARCHAR2(100),
      last_name VARCHAR2(100)
   );
```

```
    TYPE player_tab_type IS TABLE OF player_rec_type;

    v_players player_tab_type;
BEGIN
    SELECT p.Player_ID, p.First_Name, p.Last_Name
    BULK COLLECT INTO v_players
    FROM Players p
    JOIN Belongs_To bt ON p.Player_ID = bt.Player_ID
    JOIN Can_Make cm ON bt.Team_ID = cm.Team_ID
    JOIN Booking b ON cm.Booking_ID = b.Booking_ID
    WHERE b.Status = 'Confirmed';

    IF v_players.COUNT = 0 THEN
        DBMS_OUTPUT.PUT_LINE('No players with confirmed bookings found.');
    ELSE
        FOR i IN 1..v_players.COUNT LOOP
            DBMS_OUTPUT.PUT_LINE('Player Name: ' || v_players(i).first_name || ' ' ||
v_players(i).last_name);
        END LOOP;
    END IF;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No confirmed booking records found.');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error retrieving confirmed booking players: ' || SQLERRM);
```

END;

/



Question-2: Use a cursor-based record to print the player names and their associated team names.

Answer:

-- Project: Outdoor Sports Management System

-- Semester: Summer 2025

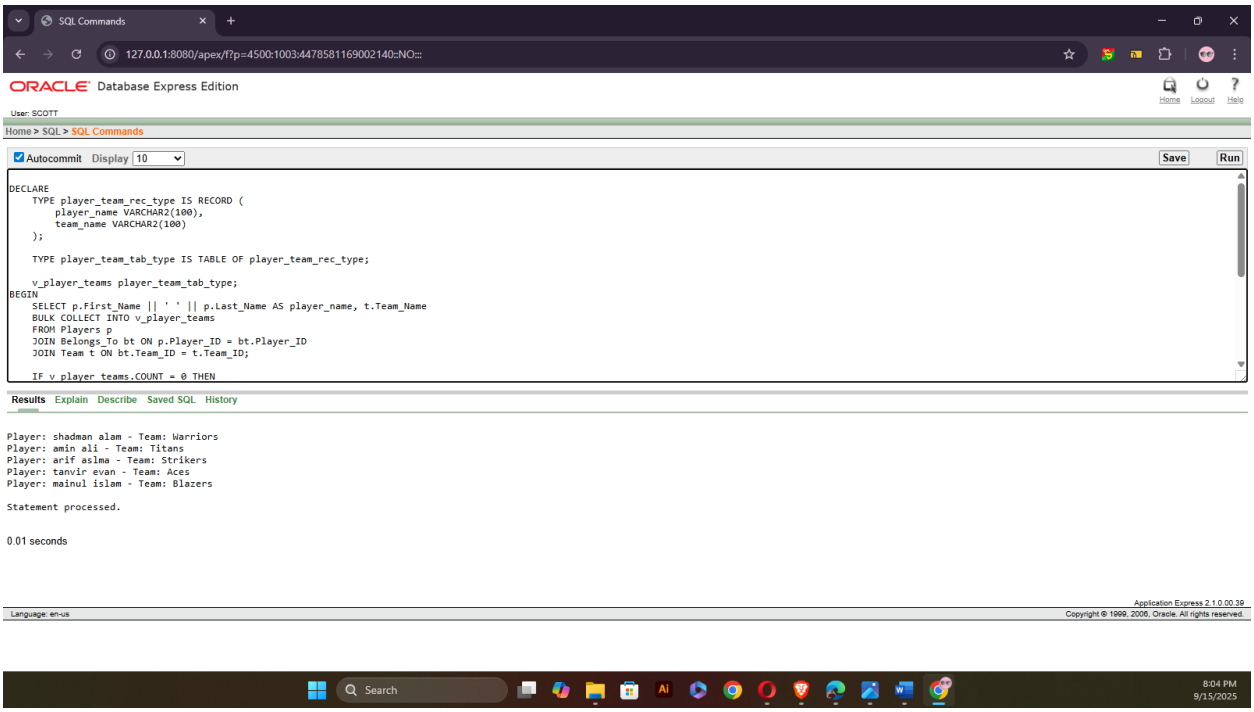
-- Course Name: Advance Database Management Systems

-- Section: A

```
DECLARE
  TYPE player_team_rec_type IS RECORD (
    player_name VARCHAR2(100),
    team_name VARCHAR2(100)
  );
```

```
  TYPE player_team_tab_type IS TABLE OF player_team_rec_type;

  v_player_teams player_team_tab_type;
BEGIN
  SELECT p.First_Name || ' ' || p.Last_Name AS player_name, t.Team_Name
  BULK COLLECT INTO v_player_teams
  FROM Players p
  JOIN Belongs_To bt ON p.Player_ID = bt.Player_ID
  JOIN Team t ON bt.Team_ID = t.Team_ID;

  IF v_player_teams.COUNT = 0 THEN
    DBMS_OUTPUT.PUT_LINE('No player-team records found.');
  ELSE
    FOR i IN 1..v_player_teams.COUNT LOOP
      DBMS_OUTPUT.PUT_LINE('Player: ' || v_player_teams(i).player_name ||
                           ' - Team: ' || v_player_teams(i).team_name);
    END LOOP;
  END IF;

EXCEPTION
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('No player-team records available.');
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Error retrieving player-team records: ' || SQLERRM);
END;
```

/



6. Row level trigger

Question-1: Create a row-level trigger that automatically sets the booking status to 'Pending' when a new booking record is inserted.

Answer:

-- Project: Outdoor Sports Management System

-- Semester: Summer 2025

-- Course Name: Advance Database Management Systems

-- Section: A

CREATE OR REPLACE TRIGGER trg_booking_default_status

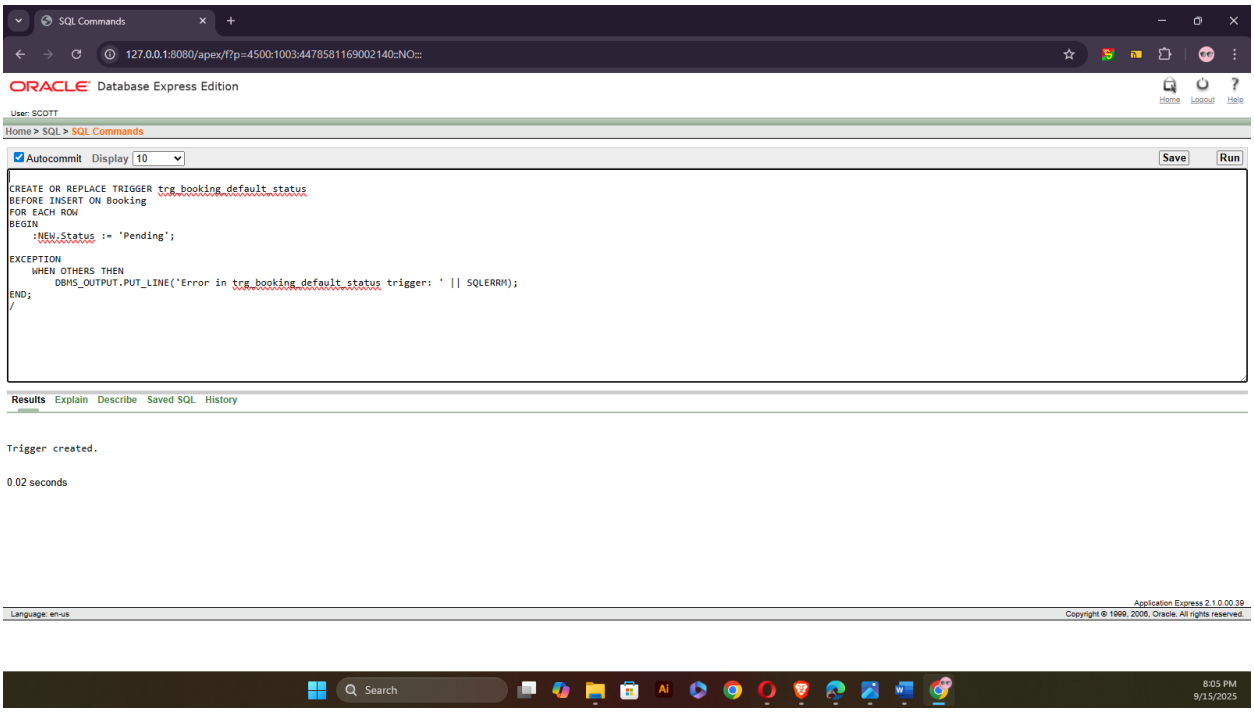
BEFORE INSERT ON Booking

FOR EACH ROW

BEGIN

:NEW.Status := 'Pending';

```
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error in trg_booking_default_status trigger: ' || SQLERRM);
END;
/
```

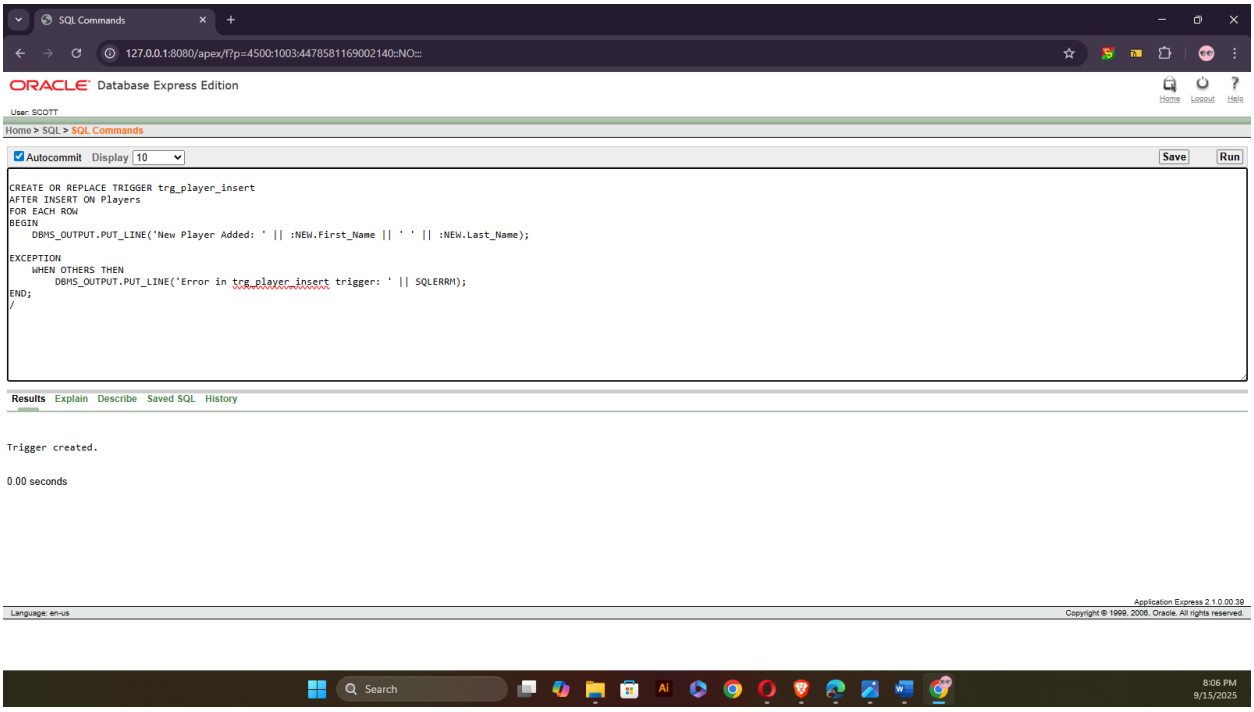


Question-2: Write a **row-level trigger** on the Players table that displays the name of a newly inserted player.

Answer:

```
-- Project: Outdoor Sports Management System
-- Semester: Summer 2025
-- Course Name: Advance Database Management Systems
-- Section: A


CREATE OR REPLACE TRIGGER trg_player_insert
AFTER INSERT ON Players
```

```
FOR EACH ROW
BEGIN
   DBMS_OUTPUT.PUT_LINE('New Player Added: ' || :NEW.First_Name || ' ' ||
:NEW.Last_Name);


EXCEPTION
   WHEN OTHERS THEN
      DBMS_OUTPUT.PUT_LINE('Error in trg_player_insert trigger: ' || SQLERRM);
END;
/
```



7. Statement level trigger

Question-1: Write a statement-level trigger that prints a message whenever a new event is inserted into the Events table.

Answer:

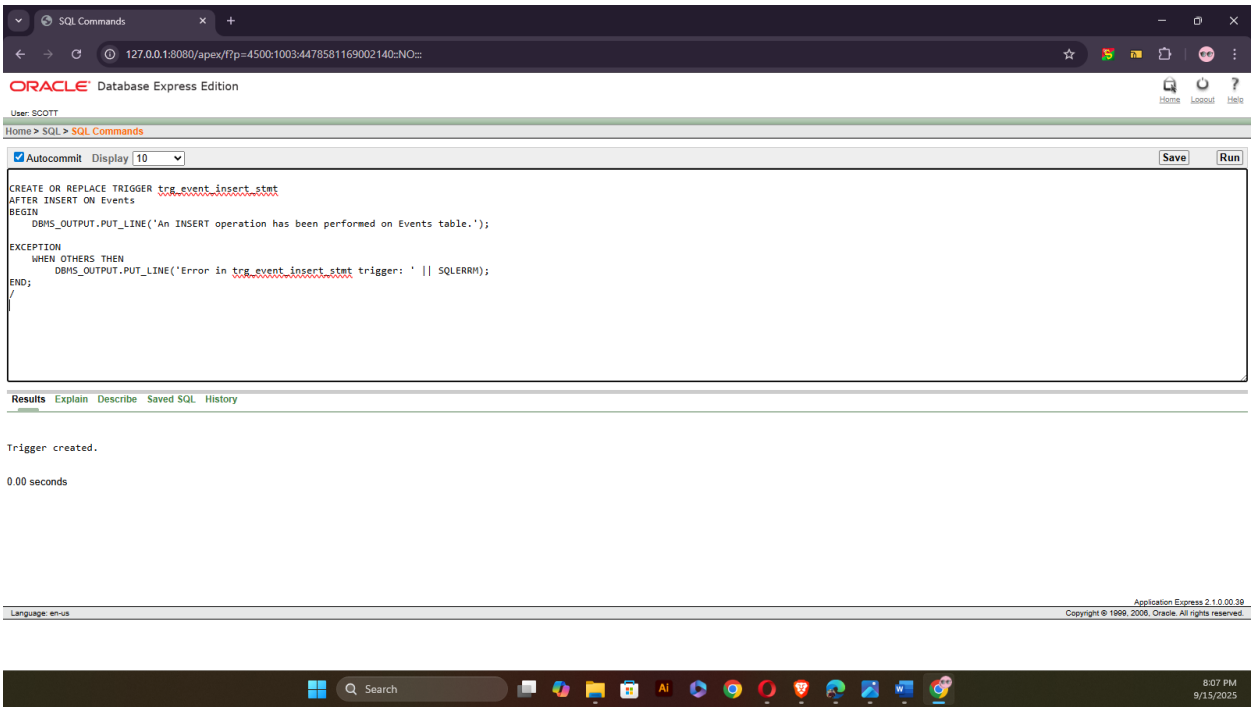
```
-- Project: Outdoor Sports Management System
```

-- Semester: Summer 2025

-- Course Name: Advance Database Management Systems

-- Section: A

```
CREATE OR REPLACE TRIGGER trg_event_insert_stmt
AFTER INSERT ON Events
BEGIN
   DBMS_OUTPUT.PUT_LINE('An INSERT operation has been performed on Events table.');

EXCEPTION
   WHEN OTHERS THEN
      DBMS_OUTPUT.PUT_LINE('Error in trg_event_insert_stmt trigger: ' || SQLERRM);
END;
/
```



Question-2: Write a statement-level trigger that prints a message before any row is deleted from the Team table

Answer:

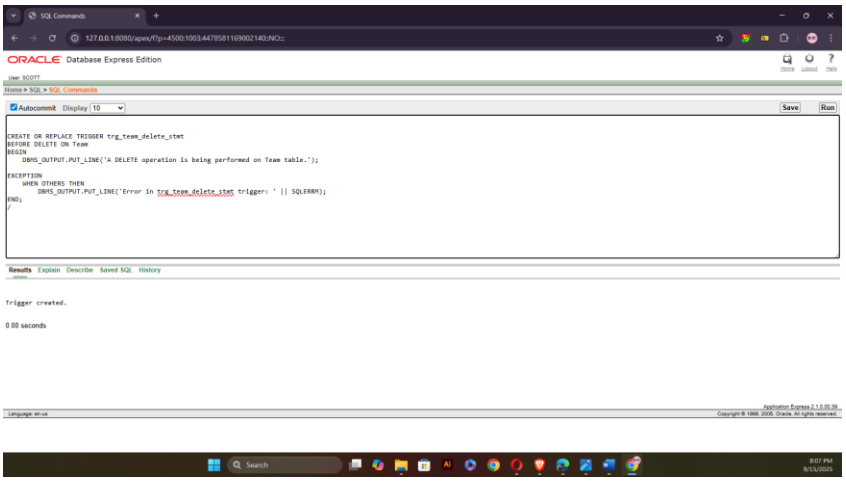
-- Project: Outdoor Sports Management System

-- Semester: Summer 2025

-- Course Name: Advance Database Management Systems

-- Section: A

```
CREATE OR REPLACE TRIGGER trg_team_delete_stmt
BEFORE DELETE ON Team
BEGIN
  DBMS_OUTPUT.PUT_LINE('A DELETE operation is being performed on Team table.');

EXCEPTION
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Error in trg_team_delete_stmt trigger: ' || SQLERRM);
END;
/
```



8. package

Question-1: Create a PL/SQL package named player_pack that contains procedures to display a player's full name and date of birth from the Players table.

Answer:

-- Project: Outdoor Sports Management System

-- Semester: Summer 2025

-- Course Name: Advance Database Management Systems

-- Section: A

Specification

```
CREATE OR REPLACE PACKAGE player_pack AS
  PROCEDURE show_player_name(p_id Players.Player_ID%TYPE);
  PROCEDURE show_player_dob(p_id Players.Player_ID%TYPE);
END player_pack;
/
```

-- Project: Outdoor Sports Management System

-- Semester: Summer 2025

-- Course Name: Advance Database Management Systems

-- Section: A

Body

```
CREATE OR REPLACE PACKAGE BODY player_pack AS

  PROCEDURE show_player_name(p_id Players.Player_ID%TYPE) IS
    p_name VARCHAR2(100);
  BEGIN
    SELECT First_Name || ' ' || Last_Name INTO p_name
    FROM Players WHERE Player_ID = p_id;
```

```
  DBMS_OUTPUT.PUT_LINE('Player Name: ' || p_name);

 EXCEPTION
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('No player found with ID: ' || p_id);
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Error in show_player_name: ' || SQLERRM);
 END;

 PROCEDURE show_player_dob(p_id Players.Player_ID%TYPE) IS
  p_dob DATE;
 BEGIN
  SELECT DOB INTO p_dob
  FROM Players WHERE Player_ID = p_id;

  DBMS_OUTPUT.PUT_LINE('Date of Birth: ' || TO_CHAR(p_dob,'DD-MON-YYYY'));

 EXCEPTION
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('No player found with ID: ' || p_id);
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Error in show_player_dob: ' || SQLERRM);
 END;

END player_pack;
```

/

-- Project: Outdoor Sports Management System

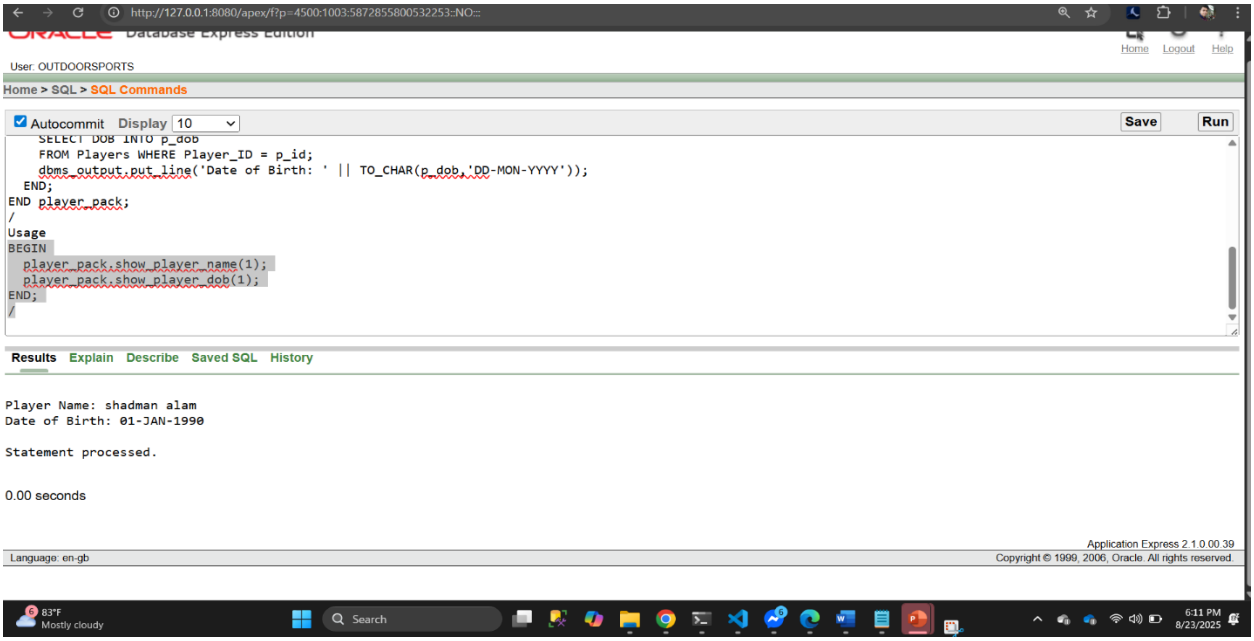
-- Semester: Summer 2025

-- Course Name: Advance Database Management Systems

-- Section: A

Usage

```
BEGIN
 player_pack.show_player_name(1);
 player_pack.show_player_dob(1);
END;
/
```



Question-2: Create a PL/SQL package named booking_pack that allows checking and updating the booking status in the Booking table.

Answer:

-- Project: Outdoor Sports Management System

-- Semester: Summer 2025

-- Course Name: Advance Database Management Systems

-- Section: A

Specification

```
CREATE OR REPLACE PACKAGE booking_pack AS

  PROCEDURE show_booking_status(b_id Booking.Booking_ID%TYPE);

  PROCEDURE update_booking_status(b_id Booking.Booking_ID%TYPE, new_status
VARCHAR2);

END booking_pack;

/


-- Project: Outdoor Sports Management System

-- Semester: Summer 2025

-- Course Name: Advance Database Management Systems

-- Section: A
```

Body

```
CREATE OR REPLACE PACKAGE BODY booking_pack AS


  PROCEDURE show_booking_status(b_id Booking.Booking_ID%TYPE) IS

    v_status VARCHAR2(50);

  BEGIN

    SELECT Status INTO v_status

    FROM Booking WHERE Booking_ID = b_id;


    DBMS_OUTPUT.PUT_LINE('Booking Status: ' || v_status);


  EXCEPTION
```

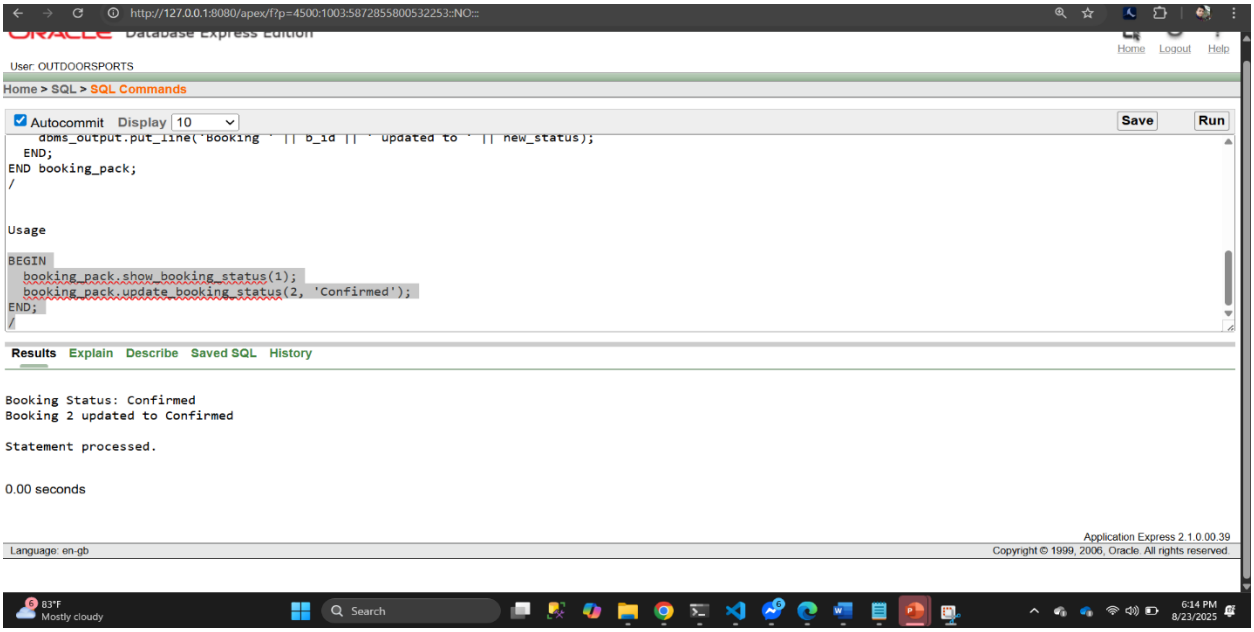
```
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('No booking found with ID: ' || b_id);
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Error in show_booking_status: ' || SQLERRM);
 END;

 PROCEDURE update_booking_status(b_id Booking.Booking_ID%TYPE, new_status
VARCHAR2) IS
 BEGIN
  UPDATE Booking SET Status = new_status WHERE Booking_ID = b_id;

  IF SQL%ROWCOUNT = 0 THEN
    DBMS_OUTPUT.PUT_LINE('No booking found with ID: ' || b_id);
  ELSE
    DBMS_OUTPUT.PUT_LINE('Booking ' || b_id || ' updated to ' || new_status);
  END IF;

 EXCEPTION
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Error in update_booking_status: ' || SQLERRM);
 END;

END booking_pack;
/
-- Project: Outdoor Sports Management System
-- Semester: Summer 2025
```

-- Course Name: Advance Database Management Systems

-- Section: A

Usage

```
BEGIN
 booking_pack.show_booking_status(1);
 booking_pack.update_booking_status(2, 'Confirmed');
END;
/
```



5. RELATIONAL ALGEBRA:

Question 1:
Find the name and location of turfs that are available for booking.

Answer:

$\pi_{\text{Turf\_Name, Location}} (\sigma_{\text{Status=”Available”}} (\text{Turf}))$

Question 2:
Find all players who have made a booking.

Answer:

$\pi_{\text{First\_Name, Last\_Name}} (\text{Players} \bowtie \text{Makes} \bowtie \text{Booking})$

Question 3:
Retrieve the team names and the names of players in each team.

Answer:

$\pi_{\text{Team\_Name, First\_Name, Last\_Name}} (\text{Team} \bowtie \text{Participates} \bowtie \text{Players})$

Question 4:
List all booking details for a specific date.

Answer:

$\pi_{\text{Booking\_ID, Start\_Time, End\_Time}} (\sigma_{\text{Date=”YYYY-MM-DD”}}(\text{Booking}))$

Question 5:
Find all players who have a skill level of 'Advanced'.

Answer:

$\pi_{\text{First\_Name, Last\_Name}} (\text{Players} \bowtie \text{Has\_Profile} \bowtie \text{Profile})$

$\sigma_{\text{Skill\_Level=”Advanced”}} (\text{Profile})$

-Thank You-